Лекция 1: Numpy, матрицы и базовые понятия в аналитике данных.

Python – очень популярный и высокоуровневый язык программирования общего назначения, который вам всем прекрасно знаком. Нельзя не упомянуть ряд его преимуществ – простой синтаксис, динамическая типизация и универсальность. Настолько он универсален, что помимо разработки и создания, скажем, веб-приложения с помощью фреймворка Django, он активно используется в аналитике и машинном обучении. Ещё он предоставляет огромную базу различных библиотек, включая те, которые понадобятся нам на этом дисциплине.

1. Numpy и ключевые понятия в базовой аналитике.

Библиотека, которая часто используется в контексте больших данных – numpy. **Numpy** - популярная библиотека с открытым исходным кодом для языка программирования Python, предназначенная для работы с многомерными массивами и матрицами, превосходит стандартные списки Python по скорости математических операций и предоставляет множество встроенных функций и методов, которые мы можем использовать на своих наборах данных. С помощью этой библиотеки мы будем рассматривать основные понятия в аналитике данных.

Для того, чтобы импортировать библиотеку используется команда **import.** Для того, чтобы удобно обращаться к библиотеке и каждый раз не прописывать ее полное название при вызове функции, можно ключевым словом as выдать ей псевдоним. В случае с numpy общепринятым считается «np».

```
[1]: import numpy as np
```

С помощью функции **np.array** можно формировать одномерные массивы из списков, рассмотрим на примере набора данных зарплат сотрудников, с этим набором данных будем работать в данной лекции:

```
[2]: salaries = np.array([40, 30, 56, 89, 100, 120, 400, 76, 80, 90, 200])
```

Например, нам понадобилось узнать среднее значение по зарплатам сотрудников.

Среднее значение - числовая характеристика, равная сумме всех чисел набора, деленной на их общее количество, соответственно мы бы могли сложить все значения в наборе данных зарплат и поделить на их общее количество, но в numpy есть функция,

которая сделает это за нас – **mean**, в качестве параметра принимает набор данных, по которому нужно найти среднее.

```
:  print('Среднее значение: ', np.mean(salaries))
```

```
Среднее значение:  116.45454545454545
```

Значение можно округлить с помощью **round** - встроенная функция Python. Она округляет число с плавающей точкой до той цифры, которую задает пользователь (то есть в качестве параметров функции указываем сначала число, которое надо округлить, а потом число знаков после запятой), например:

```
7]:  print('Среднее значение: ', round(np.mean(salaries), 1))
```

```
Среднее значение:  116.5
```

Медиана - срединное значение в наборе данных, которое делит весь ряд пополам. Ровно половина чисел в отсортированном списке больше медианы, а вторая половина - меньше. Найти медиану можно с помощью функции median из библиотеки numpy:

```
]:  print('Медиана: ', np.median(salaries))
```

```
Медиана:  89.0
```

Медиана ищется в отсортированном массиве, то есть то число, которое будет по центру в таком случае – это медиана. Если в вашем массиве четное количество элементов – медиана через numpy посчитается как среднее арифметическое значение двух чисел, стоящих в середине ряда данных. Чтобы было более наглядно:

- Для ряда {3, 5, 7, 10, 100} медиана равна 7 (это число точно посередине, весь набор данных включает в сумме 5 элементов, что является нечетным числом, поэтому медиана тут будет именно из ряда данных).
- Для ряда {3, 5, 7, 10} числа посередине - 5 и 7. Медиана равна среднему арифметическому чисел посередине, так как число элементов в этом ряду данных – 4, что является четным числом, поэтому медиана = (5 + 7)/2 = 6.

Максимальное значение – наибольшее по значению число в наборе данных, а **минимальное** – наименьшее. Ищется с помощью функций **min() и max()** из библиотеки numpy и выводит найденные значения:

```
[7]: print('Максимальное значение: ', np.max(salaries))

     Максимальное значение:  400

[8]: print('Минимальное значение: ', np.min(salaries))

     Минимальное значение:  30
```

Теперь поговорим про такое понятие, как **дисперсия**. Это мера разброса данных или случайной величины относительно их среднего значения. Если простыми словами, то дисперсия - показатель того, **насколько сильно числа в группе отличаются друг от друга и от своего среднего**. Ещё проще – дисперсия нам показывает, как сильно числа «разъехались» от среднего, то есть если числа сидят кучно и близко к среднему – дисперсия маленькая. Если числа разбежались далеко - дисперсия большая.

Чтобы её найти, можно воспользоваться функцией **var** в numpy. Но перед этим следует принять во внимание важный момент. Бывает дисперсия генеральная, а бывает выборочная. Соответственно данные тоже делятся на два этих типа. Если мы работаем со всеми существующими по этой области данными в принципе – это генеральная совокупность, если нет, то это выборочная. В аналитике, как правило, в основном работают с выборками данных и все данные во внимание берутся в редких случаях, но, тем не менее, берутся тоже. Представьте данные продаж за месяц – это только выборка, мы не можем знать все продажи в принципе, потому что тогда бы нам пришлось заглянуть в будущее и узнать все остальные продажи, что нас ожидают, а это сделать физически невозможно.

Такое отмечается особым параметром **ddof – delta degrees of freedom** (поправка на число степеней свободы), он может быть равен нулю или единице (конечно, он принимает и другие целые неотрицательные числа, но это в редких или очень узкоспециализированных ситуациях, а также в учебных примерах, нам следует запомнить 1 и 0). Здесь всё просто 0 – генеральная совокупность, 1 – выборка. Разберем, откуда берется эта единица на примере:

У нас есть уравнение из трех чисел x1, x2, x3. Представим, что мы знаем среднее, например 10. Получаем (x1, x2, x3)/3=10. Зная количество чисел и их среднее можно легко найти сумму чисел – она в этом случае равна 30. Попробуем найти x-значения:

- **x1** - может быть любым числом, оно задано свободно, никаких условий и требований к нему нет, например 5
- **x2** - тоже задано свободно, может быть любым числом, например 7.

- **x3** - уже не может быть свободным, мы знаем, что сейчас по известным и заданным свободно x1 и x2 мы получаем уравнение (7+5)+x3=30 -> 12+x3=30. Чтобы не нарушить логику и само уравнение, третье число обязано быть 30-12=18, других вариантов у нас нет.

Выходит, что два числа у нас заданы свободно, это и есть две степени свободы. А третье число не является свободным. Так и получается, **что при фиксированном среднем из N чисел свободны только N-1.** Вот эта единичка и есть та самая потерянная степень свободы - она «ушла» на то, чтобы зафиксировать среднее. При выполнении практических работ **обязательно** учитывайте этот момент в **var()** и **std().**

Вернемся к поиску дисперсии – в функцию **var** помимо набора данных помещаем ddof=1, так как мы работаем с выборкой данных, а не с генеральной совокупностью.

```
: print('Дисперсия: ', np.var(salaries, ddof=1))
```

```
Дисперсия:  10905.472727272729
```

Стандартное отклонение - показатель того, на сколько в среднем каждое число в группе отстает от общего среднего значения. Стандартное отклонение – это корень из дисперсии. Разница дисперсии и стандартного отклонения в том, в каких единицах подается это значение, дисперсия удобна для математики и расчетов, а стандартное отклонение для интерпретации результатов, для пояснений (так как измеряется в тех же единицах, что и данные). По аналогии с тем, что нам проще работать не с условными «квадратными значениями», а с привычными, когда мы снимаем с них этот квадрат. Так как дисперсия и стандартное отклонение - вещи взаимосвязанные, то правило про ddof остается в силе. Чтобы найти стандартное отклонение надо воспользоваться функцией **std()**:

```
print('Стандартное отклонение: ', np.std(salaries, ddof=1))
```

```
Стандартное отклонение:  104.42927141023597
```

Размах данных – разница между наибольшим и наименьшим значением, то есть ширина диапазона значений. Как и дисперсия, данная мера чувствительна к выбросам – аномальным значениям, выходящим за рамки «нормальных». Если в ряду, где числа были от 30 до 50, появится число 7801 – это сильно повлияет и на среднее значение, и на дисперсию, и на размах. В numpy можно воспользоваться функцией **np.ptp()** – peak to peak

– и подать ей в качестве параметра набор данных, она сама найдет максимальное и минимальное значения, после чего найдет их разницу. В случае с данными, с которыми мы работаем – максимальная зарплата 400, а минимальная 30. Получаем 400-30=370.

```
[11]: print('Размах: ', np.ptp(salaries))
```

```
Размах:  370
```

Перцентиль (центиль, процентиль) - точка на шкале, которая делит весь набор данных на 100 равных частей (0-й перцентиль – минимум, 100-й - максимум), а также статистический показатель, который указывает значение, ниже которого находится определенная доля (процент) всех наблюдений в наборе данных.

Рассмотрим на примере сдачи работы на определенный балл: если ваш балл равен 90-му процентилю, вы и какое-то другое количество людей с баллом ниже или равным вашему составляют 90% класса, оставшиеся 10% набрали балл лучший, чем у вас.

Составим список из перцентилей, которые необходимо найти. Далее с помощью функции **np.percentile**(данные, перцентиль_который_надо_найти) можно найти и их значения. В этом случае я воспользуюсь циклом for:

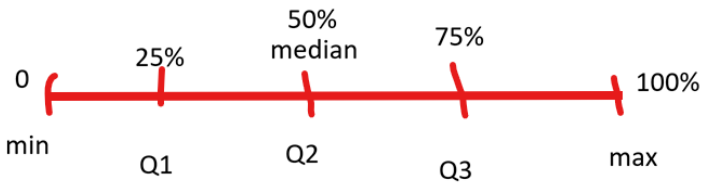
```
percentiles = [25, 70, 75, 98]
```

```
for p in percentiles:
    print(f'{p}-й перцентиль {np.percentile(salaries, p)}')
```

```
25-й перцентиль 66.0
70-й перцентиль 100.0
75-й перцентиль 110.0
98-й перцентиль 360.0000000000001
```

Квартиль – это статистический показатель, который делит упорядоченный набор данных или список научных журналов на четыре равные части (по 25%). Происходит от латинского слова quartus – четвертый. Всего квартилей три. Пятого, шестого…квартиля не существует! Понятие «четвертый квартиль» в классической статистике обычно не используется как отдельная точка деления, так как он совпадает с максимальным значением (100% данных) в наборе данных и нам не надо четыре разделителя на условном отрезке, чтобы разделить его на четыре равные части.

Удобнее это разделение квартилями представить на отрезке:



Второй квартиль – это медиана, значение, которое делит упорядоченный набор данных строго пополам.

Найти значения квартилей можно аналогично через функцию percentile, но указав во втором параметре 25-й, 50-й и 75-й перцентили. **Каждый квартиль – это перцентиль, но не каждый перцентиль – это квартиль!**

```
q1 = np.percentile(salaries, 25)
q2 = np.percentile(salaries, 50)
q3 = np.percentile(salaries, 75)

print('1-й квартиль: ', q1)
print('2-й квартиль: ', q2)
print('3-й квартиль: ', q3)
```

```
1-й квартиль:  66.0
2-й квартиль:  89.0
3-й квартиль:  110.0
```

Межквартильный размах (IQR) – разница между третьим и первым квартилем, по сравнению с размахом обычным, этот показатель очень устойчив к выбросам.

```
iqr = q3 - q1
print('межквартильный размах: ', iqr)
```

```
межквартильный размах:  44.0
```

Выбросы можно искать как раз через межквартильный размах, для этого сначала ищем границы нормы, а потом всё то, что в них не попадает, считаем за выбросы. **Границы нормы** по сути своей – диапазон наших типичных и вполне обыденных значений, например, в кофейне каждый день примерно одинаковый средний чек, но в день какого-то праздника он может резко увеличиться во много раз. Это тоже будет считаться выбросом.

Чтобы посчитать границы нормы можно воспользоваться критерием Тьюки (или правилом 1.5*IQR). Сначала рассчитывается межквартильный размах, затем для поиска нижней границы нормы нужно из первого квартиля вычесть 1,5*IQR, а для поиска верхней границы нормы – к третьему квартилю прибавить 1,5*IQR. 1,5 – это значение критерия

Джона Тьюки, он вывел его эмпирически и на основе статистического обоснования решил, что это значение будет оптимальным для вычислений.

```
low = q1-1.5*iqr
high = q3 + 1.5*iqr

print('границы нормы: ', low, ' и ', high)
```

```
границы нормы:  0.0  и  176.0
```

По примеру выше получили границы нормы [0; 176], значит всё, что выше 176 или ниже 0 – выбросы. Это логично, поскольку зарплата не может быть отрицательной – отрицательная может значить или ошибку ввода или какой-нибудь долг, который таким образом отметился в системе, а больше 176 – довольно нетипичное значение по сравнению с остальными, возможно это зарплата человека, занимающего высокую должность, или зарплата человека на нескольких должностях сразу. Выброс – не всегда ошибка.

2. Матрицы.

Матрица - прямоугольная таблица чисел или других математических элементов, упорядоченных в строках и столбцах.

$$A=\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{pmatrix} - \textit{матрица} \quad 2\times 3$$

$$B=\begin{pmatrix} b_{11} & b_{12} & \dots & b_{1n} \\ b_{21} & b_{22} & \dots & b_{2n} \\ \dots & & & \\ b_{m1} & b_{m2} & \dots & b_{mn} \end{pmatrix} - \textit{матрица} \quad m\times n$$

Возьмем одномерный массив данных чисел из 15 элементов и запишем его в переменную A. Создадим новую переменную matrix и с помощью метода **reshape** (принимает в параметры число строк и число столбцов, которые будут в матрице) превратим одномерный массив в матрицу с тремя столбцами и пятью строками.

Важный момент - произведение строк на столбцы должно точно совпадать с размером исходного массива.

```
A = np.array([210, 245, 300, 180, 260, 290, 150, 330, 220, 275, 240, 310, 190, 265, 285])
```

```
matrix = A.reshape(3,5)
print(matrix)
```

```
[[210 245 300 180 260]
 [290 150 330 220 275]
 [240 310 190 265 285]]
```

С помощью атрибута **shape** выведем размерность матрицы, а с помощью метода **size** из библиотеки numpy выведем общее количество элементов матрицы (передаем как аргумент саму матрицу).

```
print('размерность матрицы: ', matrix.shape)
```

```
размерность матрицы:  (3, 5)
```

```
print('общее количество элементов: ', np.size(matrix))
```

```
общее количество элементов:  15
```

Если мы воспользуемся методом **sum** из той же библиотеки, то сможем вывести сумму значений элементов матрицы по строкам и столбцам. В качестве двух значений он принимает саму матрицу и параметр axis (ось), если axis=1, то мы найдем сумму по строкам, а если 0, то сумму по столбцам. Таким образом, выводится значение суммы по каждой строчке и столбцу после того, как мы записали две следующие строчки кода:

```
print('сумма элементов по строкам', np.sum(matrix, axis=1))
```

```
сумма элементов по строкам [1195 1265 1290]
```

```
print('сумма элементов по столбцам', np.sum(matrix, axis=0))
```

```
сумма элементов по столбцам [740 705 820 665 820]
```

Транспонирование матрицы - математическая операция, при которой строки исходной матрицы меняются местами с её столбцами.



Транспонировать имеющуюся матрицу можно с помощью атрибута T. После транспонирования можем её вывести вместе с размерностью и увидеть, что она изменилась, теперь у нее три столбца и пять строк.

По элементам матрицы в целом можно выполнять те же действия из аналитики и статистики, с которыми мы работали выше, например, найти среднее значение элементов матрицы, найти дисперсию и так далее.

[24]:
```
matrix_T = matrix.T
print(matrix_T)
```

```
[[210 290 240]
 [245 150 310]
 [300 330 190]
 [180 220 265]
 [260 275 285]]
```

[25]:
```
print(matrix_T.shape)
```

```
(5, 3)
```

[26]:
```
print('среднее: ', np.mean(matrix))
```

```
среднее:  250.0
```